

sbatch

Section: Slurm Commands (1)
Updated: Slurm Commands
[Index](#)

NAME §

sbatch - Submit a batch script to Slurm.

SYNOPSIS §

sbatch [*OPTIONS*(0)...] [: [*OPTIONS*(*N*)...]] *script*(0) [*args*(0)...]
Option(s) define multiple jobs in a co-scheduled heterogeneous job. For more details about heterogeneous jobs see the document https://slurm.schedmd.com/heterogeneous_jobs.html

DESCRIPTION §

sbatch submits a batch script to Slurm. The batch script may be given to sbatch through a file name on the command line, or if no file name is specified, sbatch will read in a script from standard input. The batch script may contain one or more lines beginning with "#SBATCH" followed by any of the CLI options documented on this page. #SBATCH directives are read directly by Slurm, so shell-specific syntax including variable names will be read as literal text. Once the first non-comment, non-whitespace line has been reached in the script, no more #SBATCH directives will be processed. See example below.

sbatch exits immediately after the script is successfully transferred to the Slurm controller and assigned a Slurm job ID. The batch script is not necessarily granted resources immediately, it may sit in the queue of pending jobs for some time before its required resources become available.

By default both standard output and standard error are directed to a file of the name "slurm-%j.out", where the "%j" is replaced with the job allocation number. The file will be generated on the first node of the job allocation. Other than the batch script itself, Slurm does no movement of user files.

When the job allocation is granted for the batch job, Slurm runs a single copy of the batch script on one of the allocated nodes. The batch host is typically the first node in the allocation, but may be placed elsewhere due to the **--batch** option.

The following document describes the influence of various options on the allocation of cpus to jobs and tasks.
https://slurm.schedmd.com/cpu_management.html

RETURN VALUE §

sbatch will return 0 on success or error code on failure.

SCRIPT PATH RESOLUTION §

The batch script is resolved in the following order:

1. If script starts with ".", then path is constructed as: current working directory / script
2. If script starts with a "/", then path is considered absolute.
3. If script is in current working directory.
4. If script can be resolved through PATH. See **path_resolution**(7).

Current working directory is the calling process working directory unless the **--chdir** argument is passed, which will override the current working directory.

OPTIONS §

- A, --account=<account>**
Charge resources used by this job to specified account. The *account* is an arbitrary string. The account name may be changed after job submission using the **scontrol** command.
- acctg-freq=<datatype>=<interval>[,<datatype>=<interval>...]**
Define the job accounting and profiling sampling intervals in seconds. This can be used to override the *JobAcctGatherFrequency* parameter in the slurm.conf file. *<datatype>=<interval>* specifies the task sampling interval for the jobacct_gather plugin or a sampling interval for a profiling type by the acct_gather_profile plugin. Multiple comma-separated *<datatype>=<interval>* pairs may be specified. Supported *datatype* values are:

task
Sampling interval for the jobacct_gather plugins and for task profiling by the acct_gather_profile plugin.
NOTE: This frequency is used to monitor memory usage. If memory limits are enforced, the highest frequency a user can request is what is configured in the slurm.conf file. It can not be disabled.

energy
Sampling interval for energy profiling using the acct_gather_energy plugin.

network
Sampling interval for infiniband profiling using the acct_gather_interconnect plugin.

filesystem
Sampling interval for filesystem profiling using the acct_gather_filesystem plugin.
- The default value for the task sampling interval is 30 seconds. The default value for all other intervals is 0. An interval of 0 disables sampling of the specified type. If the task sampling interval is 0, accounting information is collected only at job termination (reducing Slurm interference with the job).
Smaller (non-zero) values have a greater impact upon job performance, but a value of 30 seconds is not likely to be noticeable for applications having less than 10,000 tasks.
- a, --array=<indexes>**
Submit a job array, multiple jobs to be executed with identical parameters. The *indexes* specification identifies what array index values should be used. Multiple values may be specified using a comma separated list and/or a range of values with a "-" separator. For example, "--array=0-15" or "--array=0,6,16-32". A step function can also be specified with a suffix containing a colon and number. For example, "--array=0-15:4" is equivalent to "--array=0,4,8,12". A maximum number of simultaneously running tasks from the job array may be specified using a "%" separator. For example "--array=0-15%4" will limit the number of simultaneously running tasks from this job array to 4. The minimum index value is 0. the maximum value is one less than the configuration parameter MaxArraySize. **NOTE:** Currently, federated job arrays only run on the local cluster.
- batch=<list>**
Nodes can have **features** assigned to them by the Slurm administrator. Users can specify which of these **features** are required by their batch script using this options. For example a job's allocation may include both Intel Haswell and KNL nodes with features "haswell" and "knl" respectively. On such a configuration the batch script would normally benefit by executing on a faster Haswell node. This would be specified using the option "--batch=haswell". The specification can include AND and OR operators using the ampersand and vertical bar separators. For example: "--batch=haswell|broadwell" or "--batch=haswell|big_memory". The --batch argument must be a subset of the job's **--constraint=<list>** argument (i.e. the job can not request only KNL nodes, but require the script to execute on a Haswell node). If the request can not be satisfied from the resources allocated to the job, the batch script will execute on the first node of the job allocation.
- bb=<spec>**
Burst buffer specification. The form of the specification is system dependent. Also see **--bbf**. When the **--bb** option is used, Slurm parses this option and creates a temporary burst buffer script file that is used internally by the burst buffer plugins. See Slurm's burst buffer guide for more information and examples:
https://slurm.schedmd.com/burst_buffer.html
- bbf=<file_name>**
Path of file containing burst buffer specification. The form of the specification is system dependent. These burst buffer directives will be inserted into the submitted batch script. See Slurm's burst buffer guide for more information and examples:
https://slurm.schedmd.com/burst_buffer.html
- b, --begin=<time>**
Submit the batch script to the Slurm controller immediately, like normal, but tell the controller to defer the allocation of the job until the specified time.
Time may be of the form *HH:MM:SS* to run a job at a specific time of day (seconds are optional). (If that time is already past, the next day is assumed.) You may also specify *midnight*, *noon*, *elevenses* (11 AM), *fika* (3 PM) or *teatime* (4 PM) and you can have a time-of-day suffixed with *AM* or *PM* for running in the morning or the evening. You can also say what day the job will be run, by specifying a date of the form *MMDDYY* or *MM/DD/YY YYYY-MM-DD*. Combine date and time using the following format

YYYY-MM-DD[THH:MM[:SS]]. You can also give times like *now* + *count time-units*, where the time-units can be *seconds* (default), *minutes*, *hours*, *days*, or *weeks*. The keywords *today* and *tomorrow* may also be used. The value may be changed after job submission using the **scontrol** command. For example:

```
--begin=16:00
--begin=now+1hour
--begin=now+60          (seconds by default)
--begin=2010-01-20T12:34:00
```

Notes on date/time specifications:

- Although the 'seconds' field of the HH:MM:SS time specification is allowed by the code, note that the poll time of the Slurm scheduler is not precise enough to guarantee dispatch of the job on the exact second. The job will be eligible to start on the next poll following the specified time. The exact poll interval depends on the Slurm scheduler (e.g., 60 seconds with the default sched/builtin).
- If no time (HH:MM:SS) is specified, the default is (00:00:00).
- If a date is specified without a year (e.g., MM/DD) then the current year is assumed, unless the combination of MM/DD and HH:MM:SS has already passed for that year, in which case the next year is used.

-D, --chdir=<directory>

Set the working directory of the batch script to *directory* before it is executed. The path can be specified as full path or relative path to the directory where the command is executed.

--cluster-constraint=[!]<list>

Specifies features that a federated cluster must have to have a sibling job submitted to it. Slurm will attempt to submit a sibling job to a cluster if it has at least one of the specified features. If the "!" option is included, Slurm will attempt to submit a sibling job to a cluster that has none of the specified features.

-M, --clusters=<string>

Clusters to issue commands to. Multiple cluster names may be comma separated. The job will be submitted to the one cluster providing the earliest expected job initiation time. The default value is the current cluster. A value of 'all' will query to run on all clusters. Note the **--export** option to control environment variables exported between clusters. Note that the **slurmdbd** must be up for this option to work properly, unless running in a federation with **FederationParameters=fed_display** configured.

--comment=<string>

An arbitrary comment enclosed in double quotes if using spaces or some special characters.

--consolidate-segments

Ensure that all segments from the allocation will be consolidated into one higher-level aggregated block.

NOTE: This option will only work with the **topology/block** plugin.

-C, --constraint=<list>

Nodes can have **features** assigned to them by the Slurm administrator. Users can specify which of these **features** are required by their job using the constraint option. If you are looking for 'soft' constraints please see **--prefer** for more information. Only nodes having features matching the job constraints will be used to satisfy the request. Multiple constraints may be specified with AND, OR, matching OR, resource counts, etc. (some operators are not supported on all system types).

NOTE: Changeable features are features defined by a NodeFeatures plugin.

Supported **--constraint** options include:

Single Name

Only nodes which have the specified feature will be used. For example, **--constraint="intel"**

Node Count

A request can specify the number of nodes needed with some feature by appending an asterisk and count after the feature name. For example, **--nodes=16 --constraint="graphics*4"** indicates that the job requires 16 nodes and that at least four of those nodes must have the feature "graphics." If requesting more than one feature and using node counts, the request must have square brackets surrounding it.

NOTE: This option is not supported by the helpers NodeFeatures plugin. Heterogeneous jobs can be used instead.

AND

Only nodes with all of specified features will be used. The ampersand is used for an AND operator. For example, **--constraint="intel&gpu"**

OR

Only nodes with at least one of specified features will be used. The vertical bar is used for an OR operator. If changeable features are not requested, nodes in the allocation can have different features. For example, **salloc -N2 --constraint="intel|amd"** can result in a job allocation where one node has the intel feature and the other node has the amd feature. However, if the expression contains a changeable feature, then all OR operators are automatically treated as Matching OR so that all nodes in the job allocation have the same set of features. For example, **salloc -N2 --constraint="foo|bar|baz"** The job is allocated two nodes where both nodes have foo, or bar and baz (one or both nodes could have foo, bar, and baz). The helpers NodeFeatures plugin will find the first set of node features that matches all nodes in the job allocation; these features are set as active features on the node and passed to RebootProgram (see [slurm.conf\(5\)](#)) and the helper script (see [helpers.conf\(5\)](#)). In this case, the helpers plugin uses the first of "foo" or "bar,baz" that match the two nodes in the job allocation.

Matching OR

If only one of a set of possible options should be used for all allocated nodes, then use the OR operator and enclose the options within square brackets. For example, **--constraint="[rack1|rack2|rack3|rack4]"** might be used to specify that all nodes must be allocated on a single rack of the cluster, but any of those four racks can be used.

Multiple Counts

Specific counts of multiple resources may be specified by using the AND operator and enclosing the options within square brackets. For example, **--constraint="[rack1*2&rack2*4]"** might be used to specify that two nodes must be allocated from nodes with the feature of "rack1" and four nodes must be allocated from nodes with the feature "rack2".

NOTE: This option is not supported by the helpers NodeFeatures plugin.

NOTE: Multiple Counts can cause jobs to be allocated with a non-optimal network layout.

Brackets

Brackets can be used to indicate that you are looking for a set of nodes with the different requirements contained within the brackets. For example, **--constraint="[(rack1|rack2)*1&(rack3)*2]"** will get you one node with either the "rack1" or "rack2" features and two nodes with the "rack3" feature. If requesting more than one feature and using node counts, the request must have square brackets surrounding it.

NOTE: Brackets are only reserved for **Multiple Counts** and **Matching OR** syntax. AND operators require a count for each feature inside square brackets (i.e. "[quad*2&herni*1]"). Slurm will only allow a single set of bracketed constraints per job.

NOTE: Square brackets are not supported by the helpers NodeFeatures plugin. Matching OR can be requested without square brackets by using the vertical bar character with at least one changeable feature.

Parentheses

Parentheses can be used to group like node features together. For example, **--constraint="[(knl&snc4&flat)*4&haswell*1]"** might be used to specify that four nodes with the features "knl", "snc4" and "flat" plus one node with the feature "haswell" are required. Parentheses can also be used to group operations. Without parentheses, node features are parsed strictly from left to right. For example, **--constraint="foo&bar|baz"** requests nodes with foo and bar, or baz. **--constraint="foo|bar&baz"** requests nodes with foo and baz, or bar and baz (note how baz was AND'd with everything). **--constraint="foo&(bar|baz)"** requests nodes with foo and at least one of bar or baz. **NOTE:** OR within parentheses should not be used with a KNL NodeFeatures plugin but is supported by the helpers NodeFeatures plugin.

--container=<path_to_container>

Absolute path to OCI container bundle.

--container-id=<container_id>

Unique name for OCI container.

--container-type=<container_type>

Job container type to use for job.

--contiguous

If set, then the allocated nodes must form a contiguous set.

NOTE: This option will only work with the **topology/flat** plugin. Other topology plugins modify the node ordering and prevent this option from taking effect.

-S, --core-spec=<num>

Count of Specialized Cores per node reserved by the job for system operations and not used by the application. If AllowSpecResourcesUsage is enabled a job can override the CoreSpecCount of all its allocated nodes with this option. The overridden Specialized Cores will still be reserved for system processes. The job will get an implicit **--exclusive** allocation for the rest of the Cores on the nodes, resulting in the job's processes being able to use (and being charged for) all the Cores on the nodes except for the overridden Specialized Cores. This option can not be used with the **--thread-spec** option.

NOTE: Explicitly setting a job's specialized core value implicitly sets the **--exclusive** option.

--cores-per-socket=<cores>

Restrict node selection to nodes with at least the specified number of cores per socket. See additional information under **-B** option above when task/affinity plugin is enabled.

NOTE: This option may implicitly set the number of tasks (if **-n** was not specified) as one task per requested thread.

--cpu-freq=<p1>[-p2][:p3]

Request that job steps initiated by srun commands inside this sbatch script be run at some requested frequency if possible, on the CPUs selected for the step on the compute node(s).

p1 can be [#### | low | medium | high | highm1] which will set the frequency scaling_speed to the corresponding value, and set the frequency scaling_governor to UserSpace. See below for definition of the values.

p1 can be [Conservative | OnDemand | Performance | PowerSave] which will set the scaling_governor to the corresponding value. The governor has to be in the list set by the slurm.conf option CpuFreqGovernors.

When **p2** is present, **p1** will be the minimum scaling frequency and **p2** will be the maximum scaling frequency. In that case the governor **p3** or CpuFreqDef cannot be UserSpace since it doesn't support a range.

p2 can be [#### | medium | high | highm1]. p2 must be greater than p1 and is incompatible with UserSpace governor.

p3 can be [Conservative | OnDemand | Performance | PowerSave | SchedUtil | UserSpace] which will set the governor to the corresponding value.

If **p3** is UserSpace, the frequency scaling_speed, scaling_max_freq and scaling_min_freq will be statically set to the value defined by **p1**.

Any requested frequency below the minimum available frequency will be rounded to the minimum available frequency. In the same way, any requested frequency above the maximum available frequency will be rounded to the maximum available frequency.

The **CpuFreqDef** parameter in slurm.conf will be used to set the governor in absence of **p3**. If there's no **CpuFreqDef**, the default governor will be to use the system current governor set in each cpu. Specifying a range without **CpuFreqDef** or a specific governor is therefore not allowed.

Acceptable values at present include:

- #### frequency in kilohertz
- Low the lowest available frequency
- High the highest available frequency
- HighM1

- (high minus one) will select the next highest available frequency
- Medium**
 - attempts to set a frequency in the middle of the available range
- Conservative**
 - attempts to use the Conservative CPU governor
- OnDemand**
 - attempts to use the OnDemand CPU governor (the default value)
- Performance**
 - attempts to use the Performance CPU governor
- PowerSave**
 - attempts to use the PowerSave CPU governor
- UserSpace**
 - attempts to use the UserSpace CPU governor

The following informational environment variable is set in the job step when **--cpu-freq** option is requested.

```
SLURM_CPU_FREQ_REQ
```

This environment variable can also be used to supply the value for the CPU frequency request if it is set when the 'srun' command is issued. The **--cpu-freq** on the command line will override the environment variable value. The form on the environment variable is the same as the command line. See the **ENVIRONMENT VARIABLES** section for a description of the SLURM_CPU_FREQ_REQ variable.

NOTE: This parameter is treated as a request, not a requirement. If the job step's node does not support setting the CPU frequency, or the requested value is outside the bounds of the legal frequencies, an error is logged, but the job step is allowed to continue.

NOTE: Setting the frequency for just the CPUs of the job step implies that the tasks are confined to those CPUs. If task confinement (i.e. the task/affinity TaskPlugin is enabled, or the task/cgroup TaskPlugin is enabled with "ConstrainCores=yes" set in cgroup.conf) is not configured, this parameter is ignored.

NOTE: When the step completes, the frequency and governor of each selected CPU is reset to the previous values.

NOTE: When submitting jobs with the **--cpu-freq** option with linuxproc as the ProctrackType can cause jobs to run too quickly before Accounting is able to poll for job information. As a result not all of accounting information will be present.

--cpus-per-gpu=<ncpus>

Request that *ncpus* processors be allocated per allocated GPU. Steps inheriting this value will imply --exact. Not compatible with the **--cpus-per-task** option.

-c, --cpus-per-task=<ncpus>

Advise the Slurm controller that ensuing job steps will require *ncpus* number of processors per task. Without this option, the controller will just try to allocate one processor per task.

For instance, consider an application that has 4 tasks, each requiring 3 processors. If our cluster is comprised of quad-processors nodes and we simply ask for 12 processors, the controller might give us only 3 nodes. However, by using the --cpus-per-task=3 options, the controller knows that each task requires 3 processors on the same node, and the controller will grant an allocation of 4 nodes, one for each of the 4 tasks.

--deadline=<OPT>

Remove the job if no ending is possible before this deadline (start > (deadline - time[-min])). Default is no deadline. Note that if neither **DefaultTime** nor **MaxTime** are configured on the partition the job is in, the job will need to specify some form of time limit (--time[-min]) if a deadline is to be used.

Valid time formats are:

HH:MM[:SS] [AM|PM]

MMDD[YY] or MM/DD[/YY] or MM.DD[.YY]

MM/DD[/YY]-HH:MM[:SS]

YYYY-MM-DD[THH:MM[:SS]]

now[*+count*[seconds(default)|minutes|hours|days|weeks]]

midnight, elevenses (11 AM), noon, fika (3 PM), teatime (4 PM), or tomorrow

One or more time strings may be specified (e.g., 'tomorrow18:00'). If there is a conflict between them, the last one will silently take precedence.

--delay-boot=<minutes>

Do not reboot nodes in order to satisfied this job's feature specification if the job has been eligible to run for less than this time period. If the job has waited for less than the specified period, it will use only nodes which already have the specified features. The argument is in units of minutes. A default value may be set by a system administrator using the **delay_boot** option of the **SchedulerParameters** configuration parameter in the slurm.conf file, otherwise the default value is zero (no delay).

-d, --dependency=<dependency_list>

Defer the start of this job until the specified dependencies have been satisfied. Once a dependency is satisfied, it is removed from the job. *<dependency_list>* is of the form *<type:job_id[:job_id][:type:job_id[:job_id]]>* or *<type:job_id[:job_id][?type:job_id[:job_id]]>*. All dependencies must be satisfied if the ", " separator is used. Any dependency may be satisfied if the "?" separator is used. Only one separator may be used. For instance:

```
-d afterok:20:21,afterany:23
```

means that the job can run only after a 0 return code of jobs 20 and 21 AND the completion of job 23. However:

```
-d afterok:20:21?afterany:23
```

means that any of the conditions (afterok:20 OR afterok:21 OR afterany:23) will be enough to release the job. Many jobs can share the same dependency and these jobs may even belong to different users. The value may be changed after job submission using the scontrol command. Dependencies on remote jobs are allowed in a federation. Once a job dependency fails due to the termination state of a preceding job, the dependent job will never be run, even if the preceding job is requeued and has a different termination state in a subsequent execution.

after:job_id[*[+time]*[:jobid[*+time*]]...]

After the specified jobs start or are cancelled and 'time' in minutes from job start or cancellation happens, this job can begin execution. If no 'time' is given then there is no delay after start or cancellation.

afterany:job_id[:jobid...]

This job can begin execution after the specified jobs have terminated. This is the default dependency type.

afterburstbuffer:job_id[:jobid...]

This job can begin execution after the specified jobs have terminated and any associated burst buffer stage out operations have completed.

aftercorr:job_id[:jobid...]

A task of this job array can begin execution after the corresponding task ID in the specified job has completed successfully (ran to completion with an exit code of zero). If the specified job is not an array, this is treated the same as afterok.

afternotok:job_id[:jobid...]

This job can begin execution after the specified jobs have terminated in some failed state (non-zero exit code, node failure, timed out, etc). This job must be submitted while the specified job is still active or within **MinJobAge** seconds after the specified job has ended. If the dependent job ID is not found and is on the same cluster as the job submission, the job is rejected. If the dependent job ID is not found and is on a different cluster from the job submission, the dependency is marked as failed.

afterok:job_id[:jobid...]

This job can begin execution after the specified jobs have successfully executed (ran to completion with an exit code of zero). This job must be submitted while the specified job is still active or within **MinJobAge** seconds after the specified job has ended. If the dependent job ID is not found and is on the same cluster as the job submission, the job is rejected. If the dependent job ID is not found and is on a different cluster from the job submission, the dependency is marked as failed.

singleton

This job can begin execution after any previously launched jobs sharing the same job name and user have terminated. In other words, only one job by that name and owned by that user can be running or suspended at any point in time. In a federation, a singleton dependency must be fulfilled on all clusters unless DependencyParameters=disable_remote_singleton is used in slurm.conf.

-m, --distribution={["block|cyclic|arbitrary|plane=<size>"][:["["block|cyclic|fcyclic"][:["["block|cyclic|fcyclic"]]][:{Pack|NoPack}]}

Specify alternate distribution methods for remote processes. For job allocation, this sets environment variables that will be used by subsequent srun requests and also affects which cores will be selected for job allocation.

This option controls the distribution of tasks to the nodes on which resources have been allocated, and the distribution of those resources to tasks for binding (task affinity). The first distribution method (before the first ":") controls the distribution of tasks to nodes. The second distribution method (after the first ":") controls the distribution of allocated CPUs across sockets for binding to tasks. The third distribution method (after the second ":") controls the distribution of allocated CPUs across cores for binding to tasks. The second and third distributions apply only if task affinity is enabled. The third distribution is supported only if the task/cgroup plugin is configured. The default value for each distribution type is specified by *.

Note that with select/cons_tres, the number of CPUs allocated to each socket and node may be different. Refer to the [mc_support](#) document for more information on resource allocation, distribution of tasks to nodes, and binding of tasks to CPUs.

First distribution method (distribution of tasks across nodes):

*

Use the default method for distributing tasks to nodes (block). If a topology plugin is configured and the user does not specify a distribution method, tasks are distributed following topology order rather than node index order. See **CR_NO_DIST_TOPO_BLOCK** in **SelectTypeParameters** to disable this behavior.

block

The block distribution method will distribute tasks to a node such that consecutive tasks share a node. For example, consider an allocation of three nodes each with two cpus. A four-task block distribution request will distribute those tasks to the nodes with tasks one and two on the first node, task three on the second node, and task four on the third node. Block distribution is the default behavior if the number of tasks exceeds the number of allocated nodes.

cyclic

The cyclic distribution method will distribute tasks to a node such that consecutive tasks are distributed over consecutive nodes (in a round-robin fashion). For example, consider an allocation of three nodes each with two cpus. A four-task cyclic distribution request will distribute those tasks to the nodes with tasks one and four on the first node, task two on the second node, and task three on the third node. Note that when SelectType is select/cons_tres, the same number of CPUs may not be allocated on each node. Task distribution will be round-robin among all the nodes with CPUs yet to be assigned to tasks. Cyclic distribution is the default behavior if the number of tasks is no larger than the number of allocated nodes.

plane

The tasks are distributed in blocks of size *<size>*. The size must be given or SLURM_DIST_PLANESIZE must be set. The number of tasks distributed to each node is the same as for cyclic distribution, but the taskids assigned to each node depend on the plane size. Additional distribution specifications cannot be combined with this option. For more details (including examples and diagrams), please see the [mc_support](#) document and https://slurm.schedmd.com/dist_plane.html

arbitrary

The arbitrary method of distribution will allocate processes in-order as listed in file designated by the environment variable SLURM_HOSTFILE. If this variable is listed it will override any other method specified. If not set the method will default to block. Inside the hostfile must contain at minimum the number of hosts requested and be one per line or comma separated. If specifying a task count (**-n, --ntasks=<number>**), your tasks will be laid out on the nodes in the order of the file.

NOTE: The arbitrary distribution option is supported only with the generic allocation method and cannot be used with features such as topology, CR_LLN, CR_Pack_Nodes, or pack_serial_at_end. This option is meant primarily to control a job step's task layout in an existing job allocation for the srun command.

NOTE: If the number of tasks is given and a list of requested nodes is also given, the number of nodes used from that list will be reduced to match that of the number of tasks if the number of nodes in the list is greater than the number of tasks.

Second distribution method (distribution of CPUs across sockets for binding):

Use the default method for distributing CPUs across sockets (cyclic).

block

The block distribution method will distribute allocated CPUs consecutively from the same socket for binding to tasks, before using the next consecutive socket.

cyclic

The cyclic distribution method will distribute allocated CPUs for binding to a given task consecutively from the same socket, and from the next consecutive socket for the next task, in a round-robin fashion across sockets. Tasks requiring more than one CPU will have all of those CPUs allocated on a single socket if possible.

NOTE: In nodes with hyper-threading enabled, a task not requesting full cores may be distributed across sockets. This can be avoided by specifying **--ntasks-per-core=1**, which forces tasks to allocate full cores.

fcyclic

The fcyclic distribution method will distribute allocated CPUs for binding to tasks from consecutive sockets in a round-robin fashion across the sockets. Tasks requiring more than one CPU will have each CPUs allocated in a cyclic fashion across sockets.

Third distribution method (distribution of CPUs across cores for binding):

Use the default method for distributing CPUs across cores (inherited from second distribution method).

block

The block distribution method will distribute allocated CPUs consecutively from the same core for binding to tasks, before using the next consecutive core.

cyclic

The cyclic distribution method will distribute allocated CPUs for binding to a given task consecutively from the same core, and from the next consecutive core for the next task, in a round-robin fashion across cores.

fcyclic

The fcyclic distribution method will distribute allocated CPUs for binding to tasks from consecutive cores in a round-robin fashion across the cores.

Optional control for task distribution over nodes:

Pack

Rather than evenly distributing a job step's tasks evenly across its allocated nodes, pack them as tightly as possible on the nodes. This only applies when the "block" task distribution method is used.

NoPack

Rather than packing a job step's tasks as tightly as possible on the nodes, distribute them evenly. This user option will supersede the SelectTypeParameters CR_Pack_Nodes configuration parameter.

-e, --error=<filename_pattern>

Instruct Slurm to connect the batch script's standard error directly to the file name specified in the *"filename pattern"*. By default both standard output and standard error are directed to the same file. For job arrays, the default file name is "slurm-%A_%a.out", "%A" is replaced by the job ID and "%a" with the array index. For other jobs, the default file name is "slurm-%j.out", where the "%j" is replaced by the job ID. See the **filename pattern** section below for filename specification options.

-x, --exclude=<node_name_list>

Explicitly exclude certain nodes from the resources granted to the job.

--exclusive[={user|mcs|topo}]

The job allocation can not share nodes (or topology segment with the "=topo") with other running jobs (or just other users with the "=user" option or with the "=mcs" option). If user/mcs/topo are not specified (i.e. the job allocation can not share nodes with other running jobs), the job is allocated all CPUs and GRES on all nodes in the allocation, but is only allocated as much memory as it requested. This is by design to support gang scheduling, because suspended jobs still reside in memory. To request all the memory on a node, use **--mem=0**. The default shared/exclusive behavior depends on system configuration and the partition's **OverSubscribe** option takes precedence over the job's option. **NOTE:** Since shared GRES (MPS) cannot be allocated at the same time as a sharing GRES (GPU) this option only allocates all sharing GRES and no underlying shared GRES.

NOTE: This option is mutually exclusive with **--oversubscribe**.

--export=[{ALL,}<environment_variables>|ALL|NIL|NONE]

Identify which environment variables from the submission environment are propagated to the launched application. Note that SLURM_* variables are always propagated.

--export=ALL

Default mode if **--export** is not specified. All of the user's environment will be loaded (either from the caller's environment or from a clean environment if *--get-user-env* is specified).

--export=NIL

Only SLURM_* and SPANK option variables from the user environment will be defined. User must use absolute path to the binary to be executed that will define the environment. User can not specify explicit environment variables with "NIL". Unlike NONE, NIL will not automatically create a user's environment using the *--get-user-env* mechanism.

--export=NONE

Only SLURM_* and SPANK option variables from the user environment will be defined. User must use absolute path to the binary to be executed that will define the environment. User can not specify explicit environment variables with "NONE". However, Slurm will then implicitly attempt to load the user's environment on the node where the script is being executed, as if *--get-user-env* was specified. This option is particularly important for jobs that are submitted on one cluster and execute on a different cluster (e.g. with different paths). To avoid steps inheriting environment export settings (e.g. "NONE") from sbatch command, the environment variable SLURM_EXPORT_ENV should be set to "ALL" in the job script.

--export=[ALL,]<environment_variables>

Exports all SLURM_* and SPANK option environment variables along with explicitly defined variables. Multiple environment variable names should be comma separated. Environment variable names may be specified to propagate the current value (e.g. "--export=EDITOR") or specific values may be exported (e.g. "--export=EDITOR=/bin/emacs"). If "ALL" is specified, then all user environment variables will be loaded and will take precedence over any explicitly given environment variables.

Example: **--export=EDITOR,ARG1=test**

In this example, the propagated environment will only contain the variable *EDITOR* from the user's environment, *SLURM_** environment variables, and *ARG1=test*.

Example: **--export=ALL,EDITOR=/bin/emacs**

There are two possible outcomes for this example. If the caller has the *EDITOR* environment variable defined, then the job's environment will inherit the variable from the caller's environment. If the caller doesn't have an environment variable defined for *EDITOR*, then the job's environment will use the value given by **--export**.

NOTE: NONE and *[ALL,]<environment_variables>* implicitly work as if **--get-user-env** was defined. Please see the implications of this in its respective section.

--export-file={<filename>|<fd>}

If a number between 3 and OPEN_MAX is specified as the argument to this option, a readable file descriptor will be assumed (STDIN and STDOUT are not supported as valid arguments). Otherwise a filename is assumed. Export environment variables defined in *<filename>* or read from *<fd>* to the job's execution environment. The content is one or more environment variable definitions of the form NAME=value, each separated by a null character. This allows the use of special characters in environment definitions.

--extra=<string>

An arbitrary string enclosed in single or double quotes if using spaces or some special characters.

If **SchedulerParameters=extra_constraints** is enabled, this string is used for node filtering based on the *Extra* field in each node.

-B, --extra-node-info=<sockets>[:<cores>[:<threads>]]

Restrict node selection to nodes with at least the specified number of sockets, cores per socket and/or threads per core.

NOTE: These options do not specify the resource allocation size. Each value specified is considered a minimum. An asterisk (*) can be used as a placeholder without restricting node selection for that type. The individual levels can also be specified in separate options if desired:

```
--sockets-per-node=<sockets>
--cores-per-socket=<cores>
--threads-per-core=<threads>
```

If task/affinity plugin is enabled, then specifying an allocation in this manner also results in subsequently launched tasks being bound to threads if the **-B** option specifies a thread count, otherwise an option of *cores* if a core count is specified, otherwise an option of *sockets*. If SelectType is configured to select/cons_tres, it must have a parameter of CR_Core, CR_Core_Memory, CR_Socket, or CR_Socket_Memory for this option to be honored. If not specified, the scontrol show job will display 'ReqS:C:T=*:*:*'.

NOTE: This option is mutually exclusive with **--hint**, **--threads-per-core** and **--ntasks-per-core**.

NOTE: This option may implicitly set the number of tasks (if **-n** was not specified) as one task per requested thread.

--get-user-env

This option will tell sbatch to retrieve the login environment variables for the user specified in the **--uid** option. The environment variables are retrieved by running something of this sort "su - <username> -c /usr/bin/env" and parsing the output. Be aware that any environment variables already set in sbatch's environment will take precedence over any environment variables in the user's login environment. Clear any environment variables before calling sbatch that you do not want propagated to the spawned program. If the user environment retrieval fails or times out, the job will be aborted, requeued and held.

NOTE: The explicit or implicit use of **--get-user-env** relies in the capability of being able to create PID and mount namespaces. It is very advisable to ensure that PID and mount namespace creation is available and not limited (check that **/proc/sys/user/max_[pid|mnt]_namespaces** is not 0). Although they are not strictly mandatory for **--get-user-env** to work, they ensure that there are no orphan processes left after the environment is retrieved.

--gid=<group>

If **sbatch** is run as root, and the **--gid** option is used, submit the job with *group*'s group access permissions. *group* may be the group name or the numerical group ID.

NOTE: The **--gid** argument is deprecated and will be removed in a future release.

--gpu-bind=[verbose,]<type>

Equivalent to **--tres-bind=gres/gpu:[verbose,]<type>** See **--tres-bind** for all options and documentation.

--gpu-freq=[<type>=value>[,<type=value>][,verbose]

Request that GPUs allocated to the job are configured with specific frequency values. This option can be used to independently configure the GPU and its memory frequencies. After the job is completed, the frequencies of all affected GPUs will be reset to the highest possible values. In some cases, system power caps may override the requested values. The field *type* can be "memory". If *type* is not specified, the GPU frequency is implied. The *value* field can either be "low", "medium", "high", "highm1" or a numeric value in megahertz (MHz). If the specified numeric value is not possible, a value as close as possible will be used. See below for definition of the values. The *verbose* option causes current GPU frequency information to be logged. Examples of use include "--gpu-freq=medium,memory=high" and "--gpu-freq=450".

Supported *value* definitions:

low

the lowest available frequency.

medium

attempts to set a frequency in the middle of the available range.

high

the highest available frequency.

highm1

(high minus one) will select the next highest available frequency.

-G, --gpus=[type:]<number>
Specify the total number of GPUs required for the job. An optional GPU type specification can be supplied. For example "--gpus=volta:3". See also the **--gpus-per-node**, **--gpus-per-socket** and **--gpus-per-task** options.
NOTE: The allocation has to contain at least one GPU per node, or one of each GPU type per node if types are used. Use heterogeneous jobs if different nodes need different GPU types.

--gpus-per-node=[type:]<number>
Specify the number of GPUs required for the job on each node included in the job's resource allocation. An optional GPU type specification can be supplied. For example "--gpus-per-node=volta:3". Multiple options can be requested in a comma separated list, for example: "--gpus-per-node=volta:3,kepler:1". See also the **--gpus**, **--gpus-per-socket** and **--gpus-per-task** options.
NOTE: This option is mutually exclusive with **--gres=gpu**.

--gpus-per-socket=[type:]<number>
Specify the number of GPUs required for the job on each socket included in the job's resource allocation. An optional GPU type specification can be supplied. For example "--gpus-per-socket=volta:3". Multiple options can be requested in a comma separated list, for example: "--gpus-per-socket=volta:3,kepler:1". Requires job to specify a sockets per node count (--sockets-per-node). See also the **--gpus**, **--gpus-per-node** and **--gpus-per-task** options.

--gpus-per-task=[type:]<number>
Specify the number of GPUs required for the job on each task to be spawned in the job's resource allocation. An optional GPU type specification can be supplied. For example "--gpus-per-task=volta:1". Multiple options can be requested in a comma separated list, for example: "--gpus-per-task=volta:3,kepler:1". See also the **--gpus**, **--gpus-per-socket** and **--gpus-per-node** options. This option requires an explicit task count, e.g. -n, --ntasks or "--gpus=X --gpus-per-task=Y" rather than an ambiguous range of nodes with -N, --nodes. This option will implicitly set --tres-bind=gres/gpu:per_task:<gpus_per_task>, or if multiple gpu types are specified --tres-bind=gres/gpu:per_task:<gpus_per_task_type_sum>. However, that can be overridden with an explicit --tres-bind=gres/gpu specification.

--gres=<list>
Specifies a comma-delimited list of generic consumable resources requested per node. The format for each entry in the list is "name[:type]:count". The *name* is the type of consumable resource (e.g. gpu). The *type* is an optional classification for the resource (e.g. a100). The *count* is the number of those resources with a default value of 1. The count can have a suffix of "k" or "K" (multiple of 1024), "m" or "M" (multiple of 1024 x 1024), "g" or "G" (multiple of 1024 x 1024 x 1024), "t" or "T" (multiple of 1024 x 1024 x 1024 x 1024), "p" or "P" (multiple of 1024 x 1024 x 1024 x 1024 x 1024). The specified resources will be allocated to the job on each node. The available generic consumable resources is configurable by the system administrator. A list of available generic consumable resources will be printed and the command will exit if the option argument is "help". Examples of use include "--gres=gpu:2", "--gres=gpu:kepler:2", and "--gres=help".
NOTE: This option is mutually exclusive with **--gpus-per-node**.

--gres-flags=<type>
Specify generic resource task binding options.

multiple-tasks-per-sharing Negate one-task-per-sharing. This is useful if it is set by default in SelectTypeParameters. disable-binding Negate enforce-binding. This is useful if it is set by default in SelectTypeParameters. enforce-binding The only CPUs available to the job will be those bound to the selected GRES (i.e. the CPUs identified in the gres.conf file will be strictly enforced). This option may result in delayed initiation of a job. For example a job requiring two GPUs and one CPU will be delayed until both GPUs on a single socket are available rather than using GPUs bound to separate sockets, however, the application performance may be improved due to improved communication speed. Requires the node to be configured with more than one socket and resource filtering will be performed on a per-socket basis. NOTE: This option can be set by default in SelectTypeParameters. NOTE: This option is specific to SelectType=cons_tres. NOTE: This option can give undefined results if attempting to enforce binding on multiple gres on multiple sockets. one-task-per-sharing Do not allow different tasks in to be allocated shared gres from the same sharing gres. NOTE: This flag is only enforced if shared gres are requested with --tres-per-task. NOTE: This option can be set by default with SelectTypeParameters=ONE_TASK_PER_SHARING_GRES. NOTE: This option is specific to SelectTypeParameters=MULTIPLE_SHARING_GRES_PJ	-h, --help Display help information and exit. --hint=<type> Bind tasks according to application hints. NOTE: This option implies specific values for certain related options, which prevents its use with any user-specified values for --ntasks-per-core, --cores-per-socket, --sockets-per-node, --threads-per-core or -B. These conflicting options will override --hint when specified as command line arguments. If a conflicting option is specified as an environment variable, --hint as a command line argument will take precedence. compute_bound Select settings for compute bound applications: use all cores in each socket, one thread per core. memory_bound Select settings for memory bound applications: use only one core in each socket, one thread per core. multithread Use extra threads with in-core multi-threading which can benefit communication intensive applications. Only supported with the task/affinity plugin. nomultithread Don't use extra threads with in-core multi-threading; restricts tasks to one thread per core. Only supported with the task/affinity plugin. help show this help message
-H, --hold Specify the job is to be submitted in a held state (priority of zero). A held job can now be released using scontrol to reset its priority (e.g. "scontrol release <job_id>"). --ignore-pbs Ignore all "#PBS" and "#BSUB" options specified in the batch script. -i, --input=<filename_pattern> Instruct Slurm to connect the batch script's standard input directly to the file name specified in the "<i>filename pattern</i>". By default, "/dev/null" is open on the batch script's standard input and both standard output and standard error are directed to a file of the name "slurm-%j.out", where the "%j" is replaced with the job allocation number, as described below in the filename pattern section. -J, --job-name=<jobname> Specify a name for the job allocation. The specified name will appear along with the job ID number when querying running jobs on the system. The default is the name of the batch script, or just "sbatch" if the script is read on sbatch's standard input. --kill-on-invalid-dep=<yes no> If a job has an invalid dependency and it can never run this parameter tells Slurm to terminate it or not. A terminated job state will be JOB_CANCELLED. If this option is not specified the system wide behavior applies. By default the job stays pending with reason DependencyNeverSatisfied or if the kill_invalid_depend is specified in slurm.conf the job is terminated. -L, --licenses=<license>[@db][:count][,license[@db][:count]...] Specification of licenses (or other resources available on all nodes of the cluster) which must be allocated to this job. License names can be followed by a colon and count (the default count is one). Multiple licenses can be requested. If they are separated by a comma (',' meaning AND), then all requested licenses are required for the job. For example, "--licenses=foo:4,bar". If they are separated by a pipe (' ' meaning OR), then only one of the license requests are required for the job. For example, "--licenses=foo:4 bar". AND and OR cannot both be used. To submit jobs using remote licenses (those served by slurmdbd), specify the name of the server providing the licenses. For example, "--license=nastran@slurmdb:12". If LicenseParameters=RemoteFuzzyMatch is set in slurm.conf, it is possible to omit the server name.	NOTE: When submitting heterogeneous jobs, license requests may only be made on the first component job. For example "sbatch -L ansys:2 : script.sh".

NOTE: If licenses are tracked in AccountingStorageTres and OR is used, ReqTRES will display all requested tres separated by commas. AllocTRES will display only the license that was allocated to the job.

NOTE: When a job requests OR'd licenses, Slurm will attempt to allocate the licenses in the order in which they are requested. This specified order will take precedence even if the rest of requested licenses could be satisfied on a requested reservation. This also applies to backfill planning when **SchedulerParameters=bf_licenses** is configured.

--mail-type=<type>
Notify user by email when certain event types occur. Valid *type* values are NONE, BEGIN, END, FAIL, QUEUE, ALL (equivalent to BEGIN, END, FAIL, INVALID_DEPEND, QUEUE, and STAGE_OUT), INVALID_DEPEND (dependency never satisfied), STAGE_OUT (burst buffer stage out and teardown completed), TIME_LIMIT, TIME_LIMIT_90 (reached 90 percent of time limit), TIME_LIMIT_80 (reached 80 percent of time limit), TIME_LIMIT_50 (reached 50 percent of time limit) and ARRAY_TASKS (send emails for each array task). Multiple *type* values may be specified in a comma separated list. NONE will suppress all event notifications, ignoring any other values specified. By default no email notifications are sent. The user to be notified is indicated with **--mail-user**.
Unless the ARRAY_TASKS option is specified, mail notifications on job BEGIN, END, FAIL and QUEUE apply to a job array as a whole rather than generating individual email messages for each task in the job array.

--mail-user=<user>
User to receive email notification of state changes as defined by **--mail-type**. This may be a full email address or a username. If a username is specified, the value from **MailDomain** in slurm.conf will be appended to create an email address. The default value is the submitting user.

--mcs-label=<mcs>
Used only when a compatible **MCSP** plugin is enabled. This parameter is a group that the user belongs to (**mcs/group**) or an arbitrary label string (**mcs/label**). In both cases, no label will be assigned by default. Refer to the MCS documentation for more details: <https://slurm.schedmd.com/mcs.html>

--mem=<size>[units]
Specify the real memory required per node. Default units are mebibytes. Different units can be specified using the suffix [K|M|G|T]. Default value is **DefMemPerNode** and the maximum value is **MaxMemPerNode**. If configured, both parameters can be seen using the **scontrol show config** command. This parameter would generally be used if whole nodes are allocated to jobs (**SelectType=select/linear**). Also see **--mem-per-cpu** and **--mem-per-gpu**. The **--mem**, **--mem-per-cpu** and **--mem-per-gpu** options are mutually exclusive. If **--mem**, **--mem-per-cpu** or **--mem-per-gpu** are specified as command line arguments, then they will take precedence over the environment.
NOTE: A memory size specification of zero is treated as a special case and grants the job access to all of the memory on each node.

NOTE: The memory used by each slurmd process is included in the job's total memory usage. It typically consumes between 20MiB and 200MiB, though this can vary depending on system configuration and any loaded plugins.

NOTE: Memory requests will not be strictly enforced unless Slurm is configured to use an enforcement mechanism. See **ConstrainRAMSpace** in the [cgroupr.conf\(5\)](#) man page and **OverMemoryKill** in the [slurm.conf\(5\)](#) man page for more details.

--mem-bind=[{quiet|verbose},]<type>
Bind tasks to memory. Used only when the task/affinity plugin is enabled and the NUMA memory functions are available. **Note that the resolution of CPU and memory binding may differ on some architectures**. For example, CPU binding may be performed at the level of the cores within a processor while memory binding will be performed at the level of nodes, where the definition of "nodes" may differ from system to system. By default no memory binding is performed; any task using any CPU can use any memory. This option is typically used to ensure that each task is bound to the memory closest to its assigned CPU. **The use of any type other than "none" or "local" is not recommended**.
NOTE: To have Slurm always report on the selected memory binding for all commands executed in a shell, you can enable verbose mode by setting the SLURM_MEM_BIND environment variable value to "verbose".

The following informational environment variables are set when **--mem-bind** is in use:

SLURM_MEM_BIND_LIST

SLURM_MEM_BIND_PREFER

```
SLURM_MEM_BIND_TYPE
SLURM_MEM_BIND_VERBOSE
```

See the **ENVIRONMENT VARIABLES** section for a more detailed description of the individual SLURM_MEM_BIND* variables.

Supported options include:

- help**
show this help message
- local**
Use memory local to the processor in use
- map_mem:<list>**
Bind by setting memory masks on tasks (or ranks) as specified where <list> is <numa_id_for_task_0>,<numa_id_for_task_1>,... The mapping is specified for a node and identical mapping is applied to the tasks on every node (i.e. the lowest task ID on each node is mapped to the first ID specified in the list, etc.). NUMA IDs are interpreted as decimal values unless they are preceded with '0x' in which case they interpreted as hexadecimal values. If the number of tasks (or ranks) exceeds the number of elements in this list, elements in the list will be reused as needed starting from the beginning of the list. To simplify support for large task counts, the lists may follow a map with an asterisk and repetition count. For example "map_mem:0x0f*4,0xf0*4". For predictable binding results, all CPUs for each node in the job should be allocated to the job.
- mask_mem:<list>**
Bind by setting memory masks on tasks (or ranks) as specified where <list> is <numa_mask_for_task_0>,<numa_mask_for_task_1>,... The mapping is specified for a node and identical mapping is applied to the tasks on every node (i.e. the lowest task ID on each node is mapped to the first mask specified in the list, etc.). NUMA masks are **always** interpreted as hexadecimal values. Note that masks must be preceded with a '0x' if they don't begin with [0-9] so they are seen as numerical values. If the number of tasks (or ranks) exceeds the number of elements in this list, elements in the list will be reused as needed starting from the beginning of the list. To simplify support for large task counts, the lists may follow a mask with an asterisk and repetition count. For example "mask_mem:0*4,1*4". For predictable binding results, all CPUs for each node in the job should be allocated to the job.
- no[ne]**
don't bind tasks to memory (default)
- p[refer]**
Prefer use of first specified NUMA node, but permit use of other available NUMA nodes.
- q[uiet]**
quietly bind before task runs (default)
- rank**
bind by task rank (not recommended)
- v[erbose]**
verbosely report binding before task runs

--mem-per-cpu=<size>[units]

Minimum memory required per usable allocated CPU. Default units are mebibytes. The default value is **DefMemPerCPU** and the maximum value is **MaxMemPerCPU** (see exception below). If configured, both parameters can be seen using the **scontrol show config** command. Note that if the job's **--mem-per-cpu** value exceeds the configured **MaxMemPerCPU**, then the user's limit will be treated as a memory limit per task; **--mem-per-cpu** will be reduced to a value no larger than **MaxMemPerCPU**; **--cpus-per-task** will be set and the value of **--cpus-per-task** multiplied by the new **--mem-per-cpu** value will equal the original **--mem-per-cpu** value specified by the user. This parameter would generally be used if individual processors are allocated to jobs (**SelectType=select/cons_tres**). If resources are allocated by core, socket, or whole nodes, then the number of CPUs allocated to a job may be higher than the task count and the value of **--mem-per-cpu** should be adjusted accordingly. Also see **--mem** and **--mem-per-gpu**. The **--mem**, **--mem-per-cpu** and **--mem-per-gpu** options are mutually exclusive.

NOTE: If the final amount of memory requested by a job can't be satisfied by any of the nodes configured in the partition, the job will be rejected. This could happen if **--mem-per-cpu** is used with the **--exclusive** option for a job allocation and **--mem-per-cpu** times the number of CPUs on a node is greater than the total memory of that node.

NOTE: This applies to **usable** allocated CPUs in a job allocation. This is important when more than one thread per core is configured. If a job requests --threads-per-core with fewer threads on a core than exist on the core (or --hint=nomultithread which implies --threads-per-core=1), the job will be unable to use those extra threads on the core and those threads will not be included in the memory per CPU calculation. But if the job has access to all threads on the core, those threads will be included in the memory per CPU calculation even if the job did not explicitly request those threads.

In the following examples, each core has two threads.

In this first example, two tasks can run on separate hyperthreads in the same core because --threads-per-core is not used. The third task uses both threads of the second core. The allocated memory per cpu includes all threads:

```
$ salloc -n3 --mem-per-cpu=100
salloc: Granted job allocation 17199
$ sacct -j $SLURM_JOB_ID -X -o jobid%7,reqtres%35,alloctres%35
JobID          ReqTRES          AllocTRES
-----
17199      billing=3,cpu=3,mem=300M,node=1      billing=4,cpu=4,mem=400M,node=1
```

In this second example, because of --threads-per-core=1, each task is allocated an entire core but is only able to use one thread per core. Allocated CPUs includes all threads on each core. However, allocated memory per cpu includes only the usable thread in each core.

```
$ salloc -n3 --mem-per-cpu=100 --threads-per-core=1
salloc: Granted job allocation 17200
$ sacct -j $SLURM_JOB_ID -X -o jobid%7,reqtres%35,alloctres%35
JobID          ReqTRES          AllocTRES
-----
17200      billing=3,cpu=3,mem=300M,node=1      billing=6,cpu=6,mem=300M,node=1
```

--mem-per-gpu=<size>[units]

Minimum memory required per allocated GPU. Default units are mebibytes. Different units can be specified using the suffix [K|M|G|T]. Default value is **DefMemPerGPU** and is available on both a global and per partition basis. If configured, the parameters can be seen using the **scontrol show config** and **scontrol show partition** commands. Also see **--mem**. The **--mem**, **--mem-per-cpu** and **--mem-per-gpu** options are mutually exclusive.

--mem-update=<margin>[@<delay>

Automatically reduce the job's memory limit after it has been running for the time specified by *delay*. The new limit is set to the job's current memory usage plus *margin* percent. The *margin* value is a percentage (e.g. 20 means 20% above current usage). The *delay* specifies how long after job start to sample memory usage and apply the reduction. Acceptable time formats for *delay* include "minutes", "minutes:seconds", "hours:minutes:seconds", "days-hours", "days-hours:minutes" and "days-hours:minutes:seconds". The memory limit can only be reduced, never increased. If the calculated new limit is not lower than the current limit, no change is made. A floor of 10% of the original memory request is enforced to prevent reducing the limit to an unreasonably small value. The memory usage is measured as RSS (resident physical memory) only; swap usage is not included. When **ConstrainSwapSpace** is configured, the swap limit is also reduced proportionally with the memory limit. This option works with both **--mem** and **--mem-per-cpu** jobs. For **--mem-per-cpu** jobs, the new limit is applied as a per-node value, compared against the highest per-node allocation across all nodes. This is a one-shot operation: the reduction is applied at most once per job run. On job requeue, the auto-reduction may trigger again. This option requires **SlurmctldParameters=enable_stepmgr** and **JobAcctGatherType=jobacct_gather/cgroup** to be configured.

--mincpus=<n>

Specify a minimum number of logical cpus/processors per node.

--network=<type>

Specify information pertaining to the switch or network. The interpretation of *type* is system dependent.

The **network** option is available on systems with HPE Slingshot networks. It can be used to request a job VNI (to be used for communication between job steps in a job). It also can be used to override the default network resources allocated for the job step. Multiple values may be specified in a comma-separated list.

- tcs=<class1>[:<class2>]...**
Set of traffic classes to configure for applications. Supported traffic classes are DEDICATED_ACCESS, LOW_LATENCY, BULK_DATA, and BEST_EFFORT. The traffic classes may also be specified as TC_DEDICATED_ACCESS, TC_LOW_LATENCY, TC_BULK_DATA, and TC_BEST_EFFORT.
- no_vni**
Don't allocate any VNIs for this job (even if multi-node).
- job_vni**
Allocate a job VNI for this job.
- single_node_vni**
Allocate a job VNI for this job, even if it is a single-node job.
- adjust_limits**
If set, slurmd will set an upper bound on network resource reservations by taking the per-NIC maximum resource quantity and subtracting the reserved or used values (whichever is higher) for any system network services; this is the default.
- no_adjust_limits**
If set, slurmd will calculate network resource reservations based only upon the per-resource configuration default and number of tasks in the application; it will not set an upper bound on those reservation requests based on resource usage of already-existing system network services. Setting this will mean more application launches could fail based on network resource exhaustion, but if the application absolutely needs a certain amount of resources to function, this option will ensure that.
- disable_rdzv_get**
Disable rendezvous gets in Slingshot NICs, which can improve performance for certain applications.
- nic_distribution_count=<va>**
The number of NICs the user will evenly distribute their tasks over. Defaults to the number of NICs on each node.
- def_<rsrc>=<va>**
Per-CPU reserved allocation for this resource.
- res_<rsrc>=<va>**
Per-node reserved allocation for this resource. If set, overrides the per-CPU allocation.
- max_<rsrc>=<va>**
Maximum per-node limit for this resource.
- depth=<depth>**
Multiplier for per-CPU resource allocation. Default is the number of reserved CPUs on the node.

The resources that may be requested are:

- txqs**
Transmit command queues. The default is 2 per-CPU, maximum 1024 per-node.
- tgqs**

	Target command queues. The default is 1 per-CPU, maximum 512 per-node.
eqs	Event queues. The default is 2 per-CPU, maximum 2047 per-node.
cts	Counters. The default is 1 per-CPU, maximum 2047 per-node.
ties	Trigger list entries. The default is 1 per-CPU, maximum 2048 per-node.
ptes	Portable table entries. The default is 6 per-CPU, maximum 2048 per-node.
les	List entries. The default is 16 per-CPU, maximum 16384 per-node.
acs	Addressing contexts. The default is 2 per-CPU, maximum 1022 per-node.
On systems configured with SwitchType=switch/nvidia_imex , the following options are supported:	
unique-channel-per-segment	
Instead of one channel for the entire job, allocate one channel per segment in the job. This only takes effect when topology/block is configured.	
--nice[=<i>adjustment</i>]	Run the job with an adjusted scheduling priority within Slurm. With no adjustment value the scheduling priority is decreased by 100. A negative nice value increases the priority, otherwise decreases it. The adjustment range is +/- 2147483645. Only privileged users can specify a negative adjustment.
-k, --no-kill[=off]	Do not automatically terminate a job if one of the nodes it has been allocated fails. The user will assume the responsibilities for fault-tolerance should a node fail. The job allocation will not be revoked so the user may launch new job steps on the remaining nodes in their allocation. This option does not set the SLURM_NO_KILL environment variable. Therefore, when a node fails, steps running on that node will be killed unless the SLURM_NO_KILL environment variable was explicitly set or srun calls within the job allocation explicitly requested --no-kill. Specify an optional argument of "off" to disable the effect of the SBATCH_NO_KILL environment variable.
By default Slurm terminates the entire job allocation if any node fails in its range of allocated nodes.	
--no-requeue	Specifies that the batch job should never be requeued under any circumstances (see note below). Setting this option will prevent system administrators from being able to restart the job (for example, after a scheduled downtime), recover from a node failure, or be requeued upon preemption by a higher priority job. When a job is requeued, the batch script is initiated from its beginning. Also see the --requeue option. The <i>JobRequeue</i> configuration parameter controls the default behavior on the cluster. NOTE: ForceRequeueOnFail if set as an option to the PrologFlags parameter in slurm.conf can override this setting.
-F, --nodefile=<<i>node_file</i>>	Much like --nodelist , but the list is contained in a file of name <i>node file</i> . The node names of the list may also span multiple lines in the file. Duplicate node names in the file will be ignored. The order of the node names in the list is not important; the node names will be sorted by Slurm.
-w, --nodelist=<<i>node_name_list</i>>	Request a specific list of nodes. The job will contain as many of these nodes as possible based on the resource requirements, delaying execution as needed to wait for resources to become available. If you specify a minimum node or processor count larger than can be satisfied by the supplied node list, additional resources will be allocated on other nodes as needed (unless --segment is also used). Conversely, a lower node or processor count may only require a subset of the supplied node list.
The list may be specified as a comma-separated list of nodes, a range of nodes (e.g. node[1-5,7,...]), or a filename. The nodes list will be assumed to be a filename if it contains a "/" character. Duplicate node names in the list will be ignored. The order of the node names in the list is not important; the node names will be sorted by Slurm.	
-N, --nodes=<<i>minnodes</i>>[-<i>maxnodes</i>]]<<i>size_string</i>>	Request that a minimum of <i>minnodes</i> nodes be allocated to this job. A maximum node count may also be specified with <i>maxnodes</i> . If only one number is specified, this is used as both the minimum and maximum node count. Node count can be also specified as size_string. The size_string specification identifies what nodes values should be used. Multiple values may be specified using a comma separated list or with a step function by suffix containing a colon and number values with a "-" separator. For example, "--nodes=1-15:4" is equivalent to "--nodes=1,5,9,13". The partition's node limits supersede those of the job. If a job's node limits are outside of the range permitted for its associated partition, the job will be left in a PENDING state. This permits possible execution at a later time, when the partition limit is changed. If a job node limit exceeds the number of nodes configured in the partition, the job will be rejected. Note that the environment variable SLURM_JOB_NUM_NODES will be set to the count of nodes actually allocated to the job. See the ENVIRONMENT VARIABLES section for more information. If -N is not specified, the default behavior is to allocate enough nodes to satisfy the requested resources as expressed by per-job specification options, e.g. -n , -c and --gpus . The job will be allocated as many nodes as possible within the range specified and without delaying the initiation of the job. The node count specification may include a numeric value followed by a suffix of "k" (multiplies numeric value by 1,024) or "m" (multiplies numeric value by 1,048,576). NOTE: This option cannot be used in with arbitrary distribution.
-n, --ntasks=<<i>number</i>>	sbatch does not launch tasks, it requests an allocation of resources and submits a batch script. This option advises the Slurm controller that job steps run within the allocation will launch a maximum of <i>number</i> tasks and to provide for sufficient resources. The default is one task per node, but note that the --cpus-per-task option will change this default.
--ntasks-per-core=<<i>ntasks</i>>	Request the maximum <i>ntasks</i> be invoked on each core. Meant to be used with the --ntasks option. Related to --ntasks-per-node except at the core level instead of the node level. This option will be inherited by srun. Slurm may allocate more cpus than what was requested in order to respect this option. NOTE: This option is not supported when using <i>SelectType=select/linear</i> . This value can not be greater than --threads-per-core .
--ntasks-per-gpu=<<i>ntasks</i>>	Request that there are <i>ntasks</i> tasks invoked for every GPU. This option can work in two ways: 1) either specify --ntasks in addition, in which case a type-less GPU specification will be automatically determined to satisfy --ntasks-per-gpu , or 2) specify the GPUs wanted (e.g. via --gpus or --gres) without specifying --ntasks , and the total task count will be automatically increased if necessary to allow for any calculated task count. This option will implicitly set --tres-bind=gres/gpu:single:<ntasks> , but that can be overridden with an explicit --tres-bind=gres/gpu specification. This option is not compatible with a node range (i.e. -N< <i>minnodes-maxnodes</i> >). This option is not compatible with --gpus-per-task , --gpus-per-socket , or --ntasks-per-node . This option is not supported unless <i>SelectType=cons_tres</i> is configured (either directly or indirectly on Cray systems).
--ntasks-per-node=<<i>ntasks</i>>	Request that <i>ntasks</i> be invoked on each node. If used with the --ntasks option, the --ntasks option will take precedence and the --ntasks-per-node will be treated as a <i>maximum</i> count of tasks per node. Meant to be used with the --nodes option. This is related to --cpus-per-task=ncpus , but does not require knowledge of the actual number of cpus on each node. In some cases, it is more convenient to be able to request that no more than a specific number of tasks be invoked on each node. Examples of this include submitting a hybrid MPI/OpenMP app where only one MPI "task/rank" should be assigned to each node while allowing the OpenMP portion to utilize all of the parallelism present in the node, or submitting a single setup/cleanup/monitoring job to each node of a pre-existing allocation as one step in a larger job script.
--ntasks-per-socket=<<i>ntasks</i>>	Request the maximum <i>ntasks</i> be invoked on each socket. Meant to be used with the --ntasks option. Related to --ntasks-per-node except at the socket level instead of the node level. NOTE: This option is not supported when using <i>SelectType=select/linear</i> .
--oom-kill-step[={0 1}]	Whether to kill the entire step if an OOM event is detected in any task of a step. This overwrites the "OOMKillStep" setting in TaskPluginParam from slurm.conf. When unset it will use the setting in slurm.conf. When set, a value of "0" will disable killing the entire step, while a value of "1" will enable it. This applies to the entire allocation except for the external step. Default is "1" (enabled) when the option is found with no value.
--open-mode={append truncate}	Open the output and error files using append or truncate mode as specified. The default value is specified by the system configuration parameter <i>JobFileAppend</i> .
-o, --output=<<i>filename_pattern</i>>	Instruct Slurm to connect the batch script's standard output directly to the file name specified in the " <i>filename pattern</i> ". By default both standard output and standard error are directed to the same file. For job arrays, the default file name is "slurm-%A_%a.out", "%A" is replaced by the job ID and "%a" with the array index. For other jobs, the default file name is "slurm-%j.out", where the "%j" is replaced by the job ID. See the filename pattern section below for filename specification options.
-O, --overcommit	Overcommit resources. When applied to a job allocation (not including jobs requesting exclusive access to the nodes) the resources are allocated as if only one task per node is requested. This means that the requested number of cpus per task (-c , --cpus-per-task) are allocated per node rather than being multiplied by the number of tasks. Options used to specify the number of tasks per node, socket, core, etc. are ignored.
When applied to job step allocations (the srun command when executed within an existing job allocation), this option can be used to launch more than one task per CPU. Normally, srun will not allocate more than one process per CPU. By specifying --overcommit you are explicitly allowing more than one process per CPU. However no more than MAX_TASKS_PER_NODE tasks are permitted to execute per node. NOTE: MAX_TASKS_PER_NODE is defined in the file <i>slurm.h</i> and is not a variable, it is set at Slurm build time.	
-s, --oversubscribe	The job allocation can over-subscribe resources with other running jobs. The resources to be over-subscribed can be nodes, sockets, cores, and/or hyperthreads depending upon configuration. The default over-subscribe behavior depends on system configuration and the partition's OverSubscribe option takes precedence over the job's option. This option may result in the allocation being granted sooner than if the --oversubscribe option was not set and allow higher system utilization, but application performance will likely suffer due to competition for resources. Also see the --exclusive option. NOTE: This option is mutually exclusive with --exclusive .
--parsable	Outputs only the job ID number and the cluster name if present. The values are separated by a semicolon. Errors will still be displayed. Note that --quiet suppresses the job ID along with the other informational messages.
-p, --partition=<<i>partition_names</i>>	Request a specific partition for the resource allocation. If not specified, the default behavior is to allow the slurm controller to select the default partition as designated by the system administrator. If the job can use more than one partition, specify their names in a comma separate list and the one offering earliest initiation will be used with no regard given to the partition name ordering (although higher priority partitions will be considered first). When the job is initiated, the name of the partition used will be placed first in the job record partition string.
--prefer=<<i>list</i>>	Nodes can have features assigned to them by the Slurm administrator. Users can specify which of these features are desired but not required by their job using the prefer option. This option operates independently from --constraint and will override whatever is set there if possible. When scheduling, the features in --prefer are tried first. If a node set isn't available with those features then --constraint is attempted. See --constraint for more information, this option behaves the same way.
--priority=<<i>value</i>>	Request a specific job priority. May be subject to configuration specific constraints. <i>value</i> should either be a numeric value or "TOP" (for highest possible value). Only Slurm operators and administrators can set the priority of a job.
--profile={all none <<i>type</i>>[.<<i>type</i>>...]}	Enables detailed data collection by the acct_gather_profile plugin. Detailed data are typically time-series that are stored in an HDF5 file for the job or an InfluxDB database depending on the configured plugin.
All	All data types are collected. (Cannot be combined with other values.)
None	No data types are collected. This is the default. (Cannot be combined with other values.)
Valid <i>type</i> values are:	
Energy	

Energy data is collected.

Task

Task (I/O, Memory, ...) data is collected.

Lustre

Lustre data is collected.

Network

Network (InfiniBand) data is collected.

--propagate[=*rlimit*[*,rlimit*...]]

Allows users to specify which of the modifiable (soft) resource limits to propagate to the compute nodes and apply to their jobs. If no *rlimit* is specified, then all resource limits will be propagated. The following rlimit names are supported by Slurm (although some options may not be supported on some systems):

ALL

All limits listed below (default)

NONE

No limits listed below

AS

The maximum address space (virtual memory) for a process.

CORE

The maximum size of core file

CPU

The maximum amount of CPU time

DATA

The maximum size of a process's data segment

FSIZE

The maximum size of files created. Note that if the user sets FSIZE to less than the current size of the slurmd.log, job launches will fail with a 'File size limit exceeded' error.

MEMLOCK

The maximum size that may be locked into memory

NOFILE

The maximum number of open files

NPROC

The maximum number of processes available

RSS

The maximum resident set size. Note that this only has effect with Linux kernels 2.4.30 or older or BSD.

STACK

The maximum stack size

-q, --qos=<qos>

Request a quality of service for the job, or comma separated list of QOS. If requesting a list it will be ordered based on the priority of the QOS given with the first being the highest priority. QOS values can be defined for each user/cluster/account association in the Slurm database. Users will be limited to their association's defined set of qos's when the Slurm configuration parameter, AccountingStorageEnforce, includes "qos" in its definition.

-Q, --quiet

Suppress informational messages from sbatch such as Job ID. Only errors will still be displayed.

--reboot

Force the allocated nodes to reboot before starting the job. This is only supported with some system configurations and will otherwise be silently ignored. Only root, *SlurmUser* or admins can reboot nodes.

--requeue[=*expedite*]

Specifies that the batch job should be eligible for requeuing. The job may be requeued explicitly by a system administrator, after node failure, or upon preemption by a higher priority job. When a job is requeued, the batch script is initiated from its beginning with the same job ID. Also see the **--no-requeue** option. The *JobRequeue* configuration parameter controls the default behavior on the cluster.
The optional **expedite** parameter will request that the job be immediately eligible to start again, and be scheduled with the highest possible priority. This will only happen if expedited requeues are allowed globally with the **SlurmctldParameters=enable_expedited_requeue** in *slurm.conf*.

--reservation=<reservation_names>

Allocate resources for the job from the named reservation. If the job can use more than one reservation, specify their names in a comma separate list and the one offering earliest initiation. Each reservation will be considered in the order it was requested. All reservations will be listed in scontrol/queue through the life of the job. In accounting the first reservation will be seen and after the job starts the reservation used will replace it.

--resources=<resource_names>

Specification of hierarchical resources which must be allocated to this job. Resources names can be followed by a colon and count (the default count is one). Multiple resources in Mode 1 and Mode 2 can be requested in a comma separated list but only one of Mode 3 can be requested. For example, "--resources=flat:2,natural:1".
See <https://slurm.schedmd.com/hres.html> for more information on use of Hierarchical Resource with Slurm.

--resv-ports[=*count*]

Reserve communication ports for this job. Users can specify the number of port they want to reserve. The parameter MpiParams=ports=12000-12999 must be specified in *slurm.conf*. If the number of reserved ports is zero then no ports are reserved. Used for native Cray's PMI only. This option can only be used if the slurmstepd step management is enabled. See **--stepmgr**.

--segment=<segment_size>

When a block or ring topology is used, this defines the size of the segments that will be used to create the job allocation. No requirement would be placed on all segments for a job needing to be placed within the same higher-level block.
NOTE: If the requested node count (**--nodes**) is larger than the requested segment size, it must also be evenly divisible by the segment size. If all nodes fit within a single segment, this option has no effect.

NOTE: When used in conjunction with **--odelist=<node_list>**: The requested node count must be less than or equal to the total number of unique nodes specified in the **--odelist** argument. Requesting more nodes than available unique nodes in the provided **--odelist** will result in the job being rejected by slurmctld.

--signal=[{[R|B]:}<sig_num>[@sig_time]

When a job is within *sig_time* seconds of its end time, send it the signal *sig_num*. Due to the resolution of event handling by Slurm, the signal may be sent up to 60 seconds earlier than specified. *sig_num* may either be a signal number or name (e.g. "10" or "USR1"). *sig_time* must have an integer value between 0 and 65535. By default, no signal is sent before the job's end time. If a *sig_num* is specified without any *sig_time*, the default time will be 60 seconds. Use the "B:" option to signal only the batch shell, none of the other processes will be signaled. By default all job steps will be signaled, but not the batch shell itself. Use the "R:" option to allow this job to overlap with a reservation with MaxStartDelay set. If the "R:" option is used, preemption must be enabled on the system, and if the job is preempted it will be requeued if allowed otherwise the job will be canceled. To have the signal sent at preemption time see the **send_user_signal PreemptParameter**.

--sockets-per-node=<sockets>

Restrict node selection to nodes with at least the specified number of sockets. See additional information under **-B** option above when task/affinity plugin is enabled.
NOTE: This option may implicitly set the number of tasks (if **-n** was not specified) as one task per requested thread.

--spread-job

Spread the job allocation over as many nodes as possible and attempt to evenly distribute tasks across the allocated nodes. This option disables the topology/tree plugin.

--spread-segments

Prevent nodes within the same base block from being allocated to separate segments within the same block.
NOTE: This option will only work with the **topology/block** plugin.

--stepmgr

Enable slurmstepd step management per-job if it isn't enabled system wide. This enables job steps to be managed by a single extern slurmstepd associated with the job to manage steps. This is beneficial for jobs that submit many steps inside their allocations. **PrologFlags=contain** must be set.

--switches=<count>[@max-time]

When a tree topology is used, this defines the maximum count of leaf switches desired for the job allocation and optionally the maximum time to wait for that number of switches. If Slurm finds an allocation containing more switches than the count specified, the job remains pending until it either finds an allocation with desired switch count or the time limit expires. It there is no switch count limit, there is no delay in starting the job. Acceptable time formats include "minutes", "minutes:seconds", "hours:minutes:seconds", "days-hours", "days-hours:minutes" and "days-hours:minutes:seconds". The job's maximum time delay may be limited by the system administrator using the **SchedulerParameters** configuration parameter with the **max_switch_wait** parameter option. On a dragonfly network the only switch count supported is 1 since communication performance will be highest when a job is allocate resources on one leaf switch or more than 2 leaf switches. The default max-time is the max_switch_wait SchedulerParameters.

--test-only

Validate the batch script and return an estimate of when a job would be scheduled to run given the current job queue and all the other arguments specifying the job requirements. No job is actually submitted.

--thread-spec=<num>

Count of specialized threads per node reserved by the job for system operations and not used by the application. The application will not use these threads, but will be charged for their allocation. This option can not be used with the **--core-spec** option.
NOTE: Explicitly setting a job's specialized thread value implicitly sets its **--exclusive** option, reserving entire nodes for the job.

--threads-per-core=<threads>

Restrict node selection to nodes with at least the specified number of threads per core. In task layout, use the specified maximum number of threads per core. **NOTE:** "Threads" refers to the number of processing units on each core rather than the number of application tasks to be launched per core. See additional information under **-B** option above when task/affinity plugin is enabled.
NOTE: This option may implicitly set the number of tasks (if **-n** was not specified) as one task per requested thread.

-t, --time=<time>

Set a limit on the total run time of the job allocation. If the requested time limit exceeds the partition's time limit, the job will be left in a PENDING state (possibly indefinitely). The default time limit is the partition's default time limit. When the time limit is reached, each task in each job step is sent SIGTERM followed by SIGKILL. The interval between signals is specified by the Slurm configuration parameter **KillWait**. The **OverTimeLimit** configuration parameter may permit the job to run longer than scheduled. Time resolution is one minute and second values are rounded up to the next minute.
A time limit of zero requests that no time limit be imposed. Acceptable time formats include "minutes", "minutes:seconds", "hours:minutes:seconds", "days-hours", "days-hours:minutes" and "days-hours:minutes:seconds".

--time-min=<time>

Set a minimum time limit on the job allocation. If specified, the job may have its **--time** limit lowered to a value no lower than **--time-min** if doing so permits the job to begin execution earlier than otherwise possible. The job's time limit will not be changed after the job is allocated resources. This is performed by a backfill scheduling algorithm to allocate resources otherwise reserved for higher priority jobs. Acceptable time formats include "minutes", "minutes:seconds", "hours:minutes:seconds", "days-hours", "days-hours:minutes" and "days-hours:minutes:seconds".

--tmp=<size>[units]

Specify a minimum amount of temporary disk space per node. Default units are mebibytes. Different units can be specified using the suffix [K|M|G|T].

--tres-bind=<tres>:[verbose,]<type>[+<tres>:

[verbose,]<type>...] Specify a list of tres with their task binding options. Currently gres are the only supported tres for this options. Specify gres as "gres/<gres_name>" (e.g. gres/gpu)
Example: --tres-bind=gres/gpu:verbose,map:0,1,2,3+gres/nic:closest

By default, most tres are not bound to individual tasks

Supported binding *type* options for **gres**:

https://slurm.schedmd.com/sbatch.html#bAH

8/13

closest

Bind each task to the gres(s) which are closest. In a NUMA environment, each task may be bound to more than one gres (i.e. all gres in that NUMA environment).

map:<list>

Bind by setting gres masks on tasks (or ranks) as specified where <list> is <gres_id_for_task_0>,<gres_id_for_task_1>,... gres IDs are interpreted as decimal values. If the number of tasks (or ranks) exceeds the number of elements in this list, elements in the list will be reused as needed starting from the beginning of the list. To simplify support for large task counts, the lists may follow a map with an asterisk and repetition count. For example "map:0*4,1*4". If the task/cgroup plugin is used and ConstrainDevices is set in cgroup.conf, then the gres IDs are zero-based indexes relative to the gress allocated to the job (e.g. the first gres is 0, even if the global ID is 3). Otherwise, the gres IDs are global IDs, and all gres on each node in the job should be allocated for predictable binding results.

mask:<list>

Bind by setting gres masks on tasks (or ranks) as specified where <list> is <gres_mask_for_task_0>,<gres_mask_for_task_1>,... The mapping is specified for a node and identical mapping is applied to the tasks on every node (i.e. the lowest task ID on each node is mapped to the first mask specified in the list, etc.). gres masks are always interpreted as hexadecimal values but can be preceded with an optional '0x'. To simplify support for large task counts, the lists may follow a map with an asterisk and repetition count. For example "mask:0x0*4,0xf0*4". If the task/cgroup plugin is used and ConstrainDevices is set in cgroup.conf, then the gres IDs are zero-based indexes relative to the gres allocated to the job (e.g. the first gres is 0, even if the global ID is 3). Otherwise, the gres IDs are global IDs, and all gres on each node in the job should be allocated for predictable binding results.

none

Do not bind tasks to this gres (turns off implicit binding from --tres-per-task and --gpus-per-task).

per_task:<gres_per_task>

Each task will be bound to the number of gres specified in <gres_per_task>. Tasks are preferentially assigned gres with affinity to cores in their allocation like in *closest*, though they will take any gres if they are unavailable. If no affinity exists, the first task will be assigned the first x number of gres on the node etc. Shared gres will prefer to bind one sharing device per task if possible.

single:<tasks_per_gres>

Like *closest*, except that each task can only be bound to a single gres, even when it can be bound to multiple gres that are equally close. The gres to bind to is determined by <tasks_per_gres>, where the first <tasks_per_gres> tasks are bound to the first gres available, the second <tasks_per_gres> tasks are bound to the second gres available, etc. This is basically a block distribution of tasks onto available gres, where the available gres are determined by the socket affinity of the task and the socket affinity of the gres as specified in gres.conf's *Cores* parameter.

NOTE: Shared gres binding is currently limited to per_task or none

--tres-per-task=<list>

Specifies a comma-delimited list of trackable resources required for the job on each task to be spawned in the job's resource allocation. The format for each entry in the list is "trestype[/tresname]=count". The *trestype* is the type of trackable resource requested (e.g. cpu, gres, license, etc). The *tresname* is the name of the trackable resource, as can be seen with *sacctmgr show tres*. This is required when it exists for tres types such as gres, license, etc. (e.g. gpu, gpu:a100). In order to request a license with this option, the license(s) must be defined in the **AccountingStorageTRES** parameter of slurm.conf. The *count* is the number of those resources.

The count can have a suffix of

"k" or "K" (multiple of 1024),

"m" or "M" (multiple of 1024 x 1024),

"g" or "G" (multiple of 1024 x 1024 x 1024),

"t" or "T" (multiple of 1024 x 1024 x 1024 x 1024),

"p" or "P" (multiple of 1024 x 1024 x 1024 x 1024 x 1024).

Examples:

```
--tres-per-task=cpu=4
--tres-per-task=cpu=8,license/ansys=1
--tres-per-task=gres/gpu=1
--tres-per-task=gres/gpu:a100=2
```

The specified resources will be allocated to the job on each node. The available trackable resources are configurable by the system administrator.

NOTE: This option with gres/gpu or gres/shard will implicitly set --tres-bind=gres/[gpu|shard]:per_task:<tres_per_task>, or if multiple gpu types are specified --tres-bind=gres/gpu:per_task:<gpus_per_task_type_sum>. This can be overridden with an explicit --tres-bind specification.

NOTE: Invalid TRES for --tres-per-task include bb,billing,energy,fs,mem,node,pages,vmem.

--uid=<user>

Attempt to submit and/or run a job as *user* instead of the invoking user id. The invoking user's credentials will be used to check access permissions for the target partition. User root may use this option to run jobs as a normal user in a RootOnly partition for example. If run as root, **sbatch** will drop its permissions to the uid specified after node allocation is successful. *user* may be the user name or numerical user ID.

NOTE: The **--uid** argument is deprecated and will be removed in a future release.

--usage

Display brief help message and exit.

--use-min-nodes

If a range of node counts is given, prefer the smaller count.

-v, --verbose

Increase the verbosity of sbatch's informational messages. Multiple **-v**'s will further increase sbatch's verbosity. By default only errors will be displayed.

-V, --version

Display version information and exit.

-W, --wait

Do not exit until the submitted job terminates. The exit code of the sbatch command will be the same as the exit code of the submitted job. If the job terminated due to a signal rather than a normal exit, the exit code will be set to 1. In the case of a job array, the exit code recorded will be the highest value for any task in the job array.

--wait-all-nodes=<value>

Controls when the execution of the command begins. By default the job will begin execution as soon as the allocation is made.

0

Begin execution as soon as allocation can be made. Do not wait for all nodes to be ready for use (i.e. booted).

1

Do not begin execution until all nodes are ready for use.

--wckey=<wckey>

Specify wckey to be used with job. If TrackWCKey=no (default) in the slurm.conf this value is ignored.

--wrap=<command_string>

Sbatch will wrap the specified command string in a simple "sh" shell script, and submit that script to the slurm controller. When --wrap is used, a script name and arguments may not be specified on the command line; instead the sbatch-generated wrapper script is used.

FILENAME PATTERN §

sbatch allows for a filename pattern to contain one or more replacement symbols, which are a percent sign "%" followed by a letter (e.g. %j).

\\	Do not process any of the replacement symbols.
%%	The character "%".
%A	Job array's master job allocation number.
%a	Job array ID (index) number.
%b	Job array ID (index) number modulo 10.
%J	jobid.stepid of the running job (e.g. "128.0"). The stepid is only expanded for regular steps, not for special steps like "batch" or "extern".
%j	jobid of the running job.
%N	short hostname. This will create a separate IO file per node.
%n	Node identifier relative to current job (e.g. "0" is the first node of the running job) This will create a separate IO file per node.
%r	Restart count of the running job.
%S	SLUID of the running job.
%s	stepid of the running job.
%t §	task identifier (rank) relative to current job. This will create a separate IO file per task.
%u	User name.
%x	Job name.

A number placed between the percent character and format specifier may be used to zero-pad the result in the IO filename to at minimum of specified numbers. This number is ignored if the format specifier corresponds to non-numeric data (%N for example). The maximal number is 10, if a value greater than 10 is used the result is padding up to 10 characters. Some examples of how the format string may be used for a 4 task job step with a JobID of 128 and step id of 0 are included below:

job%J.out

```
job128.0.out
job%4j.out
job0128.out
job%2j-%2t.out
job128-00.out, job128-01.out, ...
```

PERFORMANCE §

Executing **sbatch** sends a remote procedure call to **slurmctld**. If enough calls from **sbatch** or other Slurm client commands that send remote procedure calls to the **slurmctld** daemon come in at once, it can result in a degradation of performance of the **slurmctld** daemon, possibly resulting in a denial of service.

Do not run **sbatch** or other Slurm client commands that send remote procedure calls to **slurmctld** from loops in shell scripts or other programs. Ensure that programs limit calls to **sbatch** to the minimum necessary for the information you are trying to gather.

INPUT ENVIRONMENT VARIABLES §

Upon startup, sbatch will read and handle the options set in the following environment variables. The majority of these variables are set the same way the options are set, as defined above. For flag options that are defined to expect no argument, the option can be enabled by setting the environment variable without a value (empty or NULL string), the string 'yes', or a non-zero number. Any other value for the environment variable will result in the option not being set. There are a couple exceptions to these rules that are noted below.

NOTE: Environment variables will override any options set in a batch script, and command line options will override any environment variables.

- SBATCH_ACCOUNT**
Same as **-A**, **--account**
- SBATCH_ACCTG_FREQ**
Same as **--acctg-freq**
- SBATCH_ARRAY_INX**
Same as **-a**, **--array**
- SBATCH_BATCH**
Same as **--batch**
- SBATCH_CLUSTERS** or **SLURM_CLUSTERS**
Same as **--clusters**
- SBATCH_CONSTRAINT**
Same as **-C**, **--constraint**
- SBATCH_CONTAINER**
Same as **--container**.
- SBATCH_CONTAINER_ID**
Same as **--container-id**.
- SBATCH_CONTAINER_TYPE**
Same as **--container-type**.
- SBATCH_CORE_SPEC**
Same as **--core-spec**
- SBATCH_CPUS_PER_GPU**
Same as **--cpus-per-gpu**
- SBATCH_DEBUG**
Same as **-v**, **--verbose**, when set to 1, when set to 2 gives **-vv**, etc.
- SBATCH_DELAY_BOOT**
Same as **--delay-boot**
- SBATCH_DISTRIBUTION**
Same as **-m**, **--distribution**
- SBATCH_ERROR**
Same as **-e**, **--error**
- SBATCH_EXCLUSIVE**
Same as **--exclusive**
- SBATCH_EXPORT**
Same as **--export**
- SBATCH_GET_USER_ENV**
Same as **--get-user-env**
- SBATCH_GPU_BIND**
Same as **--gpu-bind**
- SBATCH_GPU_FREQ**
Same as **--gpu-freq**
- SBATCH_GPUS**
Same as **-G**, **--gpus**
- SBATCH_GPUS_PER_NODE**
Same as **--gpus-per-node**
- SBATCH_GPUS_PER_TASK**
Same as **--gpus-per-task**
- SBATCH_GRES**
Same as **--gres**
- SBATCH_GRES_FLAGS**
Same as **--gres-flags**
- SBATCH_HINT** or **SLURM_HINT**
Same as **--hint**
- SBATCH_IGNORE_PBS**
Same as **--ignore-pbs**
- SBATCH_INPUT**
Same as **-i**, **--input**
- SBATCH_JOB_NAME**
Same as **-J**, **--job-name**
- SBATCH_MEM_BIND**
Same as **--mem-bind**
- SBATCH_MEM_PER_CPU**
Same as **--mem-per-cpu**
- SBATCH_MEM_PER_GPU**
Same as **--mem-per-gpu**
- SBATCH_MEM_PER_NODE**
Same as **--mem**
- SBATCH_NETWORK**
Same as **--network**
- SBATCH_NO_KILL**
Same as **-k**, **--no-kill**
- SBATCH_NO_REQUEUE**
Same as **--no-requeue**
- SBATCH_OPEN_MODE**
Same as **--open-mode**
- SBATCH_OUTPUT**
Same as **-o**, **--output**
- SBATCH_OVERCOMMIT**
Same as **-O**, **--overcommit**
- SBATCH_PARTITION**
Same as **-p**, **--partition**
- SBATCH_POWER**
Same as **--power**
- SBATCH_PROFILE**
Same as **--profile**
- SBATCH_QOS**
Same as **--qos**

SBATCH_REQ_SWITCH
When a tree topology is used, this defines the maximum count of switches desired for the job allocation and optionally the maximum time to wait for that number of switches. See **--switches**

SBATCH_REQUEUE
Same as **--requeue**

SBATCH_RESERVATION
Same as **--reservation**

SBATCH_SEGMENT_SIZE
Same as **--segment**

SBATCH_SIGNAL
Same as **--signal**

SBATCH_SPREAD_JOB
Same as **--spread-job**

SBATCH_THREAD_SPEC
Same as **--thread-spec**

SBATCH_THREADS_PER_CORE
Same as **--threads-per-core**

SBATCH_TIMELIMIT
Same as **-t, --time**

SBATCH_TRES_BIND
Same as **--tres-bind**

SBATCH_TRES_PER_TASK
Same as **--tres-per-task**

SBATCH_USE_MIN_NODES
Same as **--use-min-nodes**

SBATCH_WAIT
Same as **-W, --wait**

SBATCH_WAIT_ALL_NODES
Same as **--wait-all-nodes**. Must be set to 0 or 1 to disable or enable the option.

SBATCH_WAIT4SWITCH
Max time waiting for requested switches. See **--switches**

SBATCH_WCKEY
Same as **--wckey**

SLURM_CONF
The location of the Slurm configuration file.

SLURM_DEBUG_FLAGS
Specify debug flags for sbatch to use. See DebugFlags in the [slurm.conf\(5\)](#) man page for a full list of flags. The environment variable takes precedence over the setting in the slurm.conf.

SLURM_EXIT_ERROR
Specifies the exit code generated when a Slurm error occurs (e.g. invalid options). This can be used by a script to distinguish application exit codes from various Slurm error conditions.

SLURM_STEP_KILLED_MSG_NODE_ID=ID
If set, only the specified node will log when the job or step are killed by a signal.

SLURM_UMASK
If defined, Slurm will use the defined *umask* to set permissions when creating the output/error files for the job.

OUTPUT ENVIRONMENT VARIABLES §

The Slurm controller will set the following variables in the environment of the batch script.

SBATCH_MEM_BIND
Set to value of the **--mem-bind** option.

SBATCH_MEM_BIND_LIST
Set to bit mask used for memory binding.

SBATCH_MEM_BIND_PREFER
Set to "prefer" if the **--mem-bind** option includes the prefer option.

SBATCH_MEM_BIND_TYPE
Set to the memory binding type specified with the **--mem-bind** option. Possible values are "none", "rank", "map_mem:", "mask_mem:" and "local".

SBATCH_MEM_BIND_VERBOSE
Set to "verbose" if the **--mem-bind** option includes the verbose option. Set to "quiet" otherwise.

SLURM_*_HET_GROUP_#
For a heterogeneous job allocation, the environment variables are set separately for each component.

SLURM_ARRAY_JOB_ID
Job array's master job ID number.

SLURM_ARRAY_TASK_COUNT
Total number of tasks in a job array.

SLURM_ARRAY_TASK_ID
Job array ID (index) number.

SLURM_ARRAY_TASK_MAX
Job array's maximum ID (index) number.

SLURM_ARRAY_TASK_MIN
Job array's minimum ID (index) number.

SLURM_ARRAY_TASK_STEP
Job array's index step size.

SLURM_CLUSTER_NAME
Name of the cluster on which the job is executing.

SLURM_CPUS_ON_NODE
Number of CPUs allocated to the batch step. **NOTE:** The **select/linear** plugin allocates entire nodes to jobs, so the value indicates the total count of CPUs on the node. For the **cons/tres** plugin, this number indicates the number of CPUs on this node allocated to the step.

SLURM_CPUS_PER_GPU
Number of CPUs requested per allocated GPU. Only set if the **--cpus-per-gpu** option is specified.

SLURM_CPUS_PER_TASK
Number of cpus requested per task. Only set if either the **--cpus-per-task** option or the **--tres-per-task=cpu=#** option is specified.

SLURM_CONTAINER
OCI Bundle for job. Only set if **--container** is specified.

SLURM_CONTAINER_ID
OCI id for job. Only set if **--container-id** is specified.

SLURM_CONTAINER_TYPE
Job container type for job. Only set if **--container-type** is specified.

SLURM_DIST_PLANESIZE
Plane distribution size. Only set for plane distributions. See **-m, --distribution**.

SLURM_DISTRIBUTION
Same as **-m, --distribution**

SLURM_EXPORT_ENV
Same as **--export**.

SLURM_GPU_BIND
Requested binding of tasks to GPU. Only set if the **--gpu-bind** option is specified.

SLURM_GPU_FREQ
Requested GPU frequency. Only set if the **--gpu-freq** option is specified.

SLURM_GPUS
Number of GPUs requested. Only set if the **-G, --gpus** option is specified.

SLURM_GPUS_ON_NODE
Number of GPUs allocated to the batch step.

SLURM_GPUS_PER_NODE
Requested GPU count per allocated node. Only set if the **--gpus-per-node** option is specified.

SLURM_GPUS_PER_SOCKET
Requested GPU count per allocated socket. Only set if the **--gpus-per-socket** option is specified.

SLURM_GTIDS
Global task IDs running on this node. Zero origin and comma separated. It is read internally by pmi if Slurm was built with pmi support. Leaving the variable set may cause problems when using external packages from within the job (Abaqus and Ansys have been known to have problems when it is set - consult the appropriate documentation for 3rd party software).

SLURM_HET_SIZE

Set to count of components in heterogeneous job.

SLURM_JOB_ACCOUNT

Account name associated of the job allocation.

SLURM_JOB_CPUS_PER_NODE

Count of CPUs available to the job on the nodes in the allocation, using the format *CPU_count*{(*xnumber_of_nodes*)}[*,CPU_count* [(*xnumber_of_nodes*)] ...]. For example: SLURM_JOB_CPUS_PER_NODE='72(x2),36' indicates that on the first and second nodes (as listed by SLURM_JOB_NODELIST) the allocation has 72 CPUs, while the third node has 36 CPUs. **NOTE:** The **select/linear** plugin allocates entire nodes to jobs, so the value indicates the total count of CPUs on allocated nodes. The **select/cons_tres** plugin allocates individual CPUs to jobs, so this number indicates the number of CPUs allocated to the job.

SLURM_JOB_DEPENDENCY

Set to value of the **--dependency** option.

SLURM_JOB_END_TIME

The UNIX timestamp for a job's projected end time.

SLURM_JOB_GPUS

The global GPU IDs of the GPUs allocated to this job. The GPU IDs are not relative to any device cgroup, even if devices are constrained with task/cgroup. Only set in batch and interactive jobs.

SLURM_JOB_ID

The ID of the job allocation.

SLURM_JOB_LICENSES

Name and count of any license(s) requested.

SLURM_JOB_NAME

Name of the job.

SLURM_JOB_NODELIST

List of nodes allocated to the job.

SLURM_JOB_NUM_NODES

Total number of nodes in the job's resource allocation.

SLURM_JOB_PARTITION

Name of the partition in which the job is running.

SLURM_JOB_QOS

Quality Of Service (QOS) of the job allocation.

SLURM_JOB_RESERVATION

Advanced reservation containing the job allocation, if any.

SLURM_JOB_SEGMENT_SIZE

The size of the segments that was used to create the job allocation. Only set if **--segment** is specified.

SLURM_JOB_START_TIME

The UNIX timestamp for a job's start time.

SLURM_JOBID

The ID of the job allocation. See **SLURM_JOB_ID**. Included for backwards compatibility.

SLURM_LOCALID

Node local task ID for the process within a job.

SLURM_MEM_PER_CPU

Same as **--mem-per-cpu**

SLURM_MEM_PER_GPU

Requested memory per allocated GPU. Only set if the **--mem-per-gpu** option is specified.

SLURM_MEM_PER_NODE

Same as **--mem**

SLURM_NETWORK

Set to the value of the **--network** option, if specified.

SLURM_NNODES

Total number of nodes in the job's resource allocation. See **SLURM_JOB_NUM_NODES**. Included for backwards compatibility.

SLURM_NODEID

ID of the nodes allocated.

SLURM_NODELIST

List of nodes allocated to the job. See **SLURM_JOB_NODELIST**. Included for backwards compatibility.

SLURM_NPROCS

Same as **SLURM_NTASKS**. Included for backwards compatibility.

SLURM_NTASKS

Set to value of the **--ntasks** option, if specified. Or, if any of the **--ntasks-per-*** options are specified, set to the number of tasks in the job.

NOTE: This is also an input variable for `srn`, so if set it will effectively set the **--ntasks** option for `srn` when called from the batch script.

SLURM_NTASKS_PER_CORE

Number of tasks requested per core. Only set if the **--ntasks-per-core** option is specified.

SLURM_NTASKS_PER_GPU

Number of tasks requested per GPU. Only set if the **--ntasks-per-gpu** option is specified.

SLURM_NTASKS_PER_NODE

Number of tasks requested per node. Only set if the **--ntasks-per-node** option is specified.

SLURM_NTASKS_PER_SOCKET

Number of tasks requested per socket. Only set if the **--ntasks-per-socket** option is specified.

SLURM_OOMKILLSTEP

Same as **--oom-kill-step**

SLURM_OVERCOMMIT

Set to **1** if **--overcommit** was specified.

SLURM_PRIO_PROCESS

The scheduling priority (nice value) at the time of job submission. This value is propagated to the spawned processes.

SLURM_PROCID

The MPI rank (or relative process ID) of the current process

SLURM_PROFILE

Same as **--profile**

SLURM_RESTART_COUNT

If the job has been restarted due to system failure or has been explicitly queued, this will be sent to the number of times the job has been restarted.

SLURM_SHARDS_ON_NODE

Number of GPU Shards available to the step on this node.

SLURM_SUBMIT_DIR

The directory from which **sbatch** was invoked.

SLURM_SUBMIT_HOST

The hostname of the computer from which **sbatch** was invoked.

SLURM_TASK_PID

The process ID of the task being started.

SLURM_TASKS_PER_NODE

Number of tasks to be initiated on each node. Values are comma separated and in the same order as SLURM_JOB_NODELIST. If two or more consecutive nodes are to have the same task count, that count is followed by "(x#)" where "#" is the repetition count. For example, "SLURM_TASKS_PER_NODE=2(x3),1" indicates that the first three nodes will each execute two tasks and the fourth node will execute one task.

SLURM_THREADS_PER_CORE

This is only set if **--threads-per-core** or **SBATCH_THREADS_PER_CORE** were specified. The value will be set to the value specified by **--threads-per-core** or **SBATCH_THREADS_PER_CORE**. This is used by subsequent `srn` calls within the job allocation.

SLURM_TOPOLOGY_ADDR

This is set only if the system has the topology/tree plugin configured. The value will be set to the names network switches which may be involved in the job's communications from the system's top level switch down to the leaf switch and ending with node name. A period is used to separate each hardware component name.

SLURM_TOPOLOGY_ADDR_PATTERN

This is set only if the system has the topology/tree plugin configured. The value will be set component types listed in SLURM_TOPOLOGY_ADDR. Each component will be identified as either "switch" or "node". A period is used to separate each hardware component type.

SLURM_TRES_PER_TASK

Set to the value of **--tres-per-task**. If **--cpus-per-task** or **--gpus-per-task** is specified, it is also set in **SLURM_TRES_PER_TASK** as if it were specified in **--tres-per-task**.

SLURMD_NODENAME

Name of the node running the job script.

EXAMPLES §

Specify a batch script by filename on the command line. The batch script specifies a 1 minute time limit for the job.

```
$ cat myscript
#!/bin/sh
#SBATCH --time=1
srn hostname |sort
```

```
$ sbatch -N4 myscript
salloc: Granted job allocation 65537

$ cat slurm-65537.out
host1
host2
host3
host4
```

Pass a batch script to sbatch on standard input:

```
$ sbatch -N4 <<EOF
> #!/bin/sh
> srun hostname |sort
> EOF
sbatch: Submitted batch job 65541

$ cat slurm-65541.out
host1
host2
host3
host4
```

To create a heterogeneous job with 3 components, each allocating a unique set of nodes:

```
$ sbatch -w node[2-3] : -w node4 : -w node[5-7] work.bash
Submitted batch job 34987
```

COPYING §

Copyright (C) 2006-2007 The Regents of the University of California. Produced at Lawrence Livermore National Laboratory (cf, DISCLAIMER).
Copyright (C) 2008-2010 Lawrence Livermore National Security.
Copyright (C) 2010-2022 SchedMD LLC.
This file is part of Slurm, a resource management program. For details, see <<https://slurm.schedmd.com/>>.

Slurm is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

Slurm is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

SEE ALSO §

[sinfo](#)(1), [sattach](#)(1), [salloc](#)(1), [squeue](#)(1), [scancel](#)(1), [scontrol](#)(1), [slurm.conf](#)(5), [sched_setaffinity](#) (2), [numa](#) (3)

Index

- [NAME](#)
- [SYNOPSIS](#)
- [DESCRIPTION](#)
- [RETURN VALUE](#)
- [SCRIPT PATH RESOLUTION](#)
- [OPTIONS](#)
- [FILENAME PATTERN](#)
- [PERFORMANCE](#)
- [INPUT ENVIRONMENT VARIABLES](#)
- [OUTPUT ENVIRONMENT VARIABLES](#)
- [EXAMPLES](#)
- [COPYING](#)
- [SEE ALSO](#)